

DATA STRUCTURE AND ALGORITHMS (UEE410)

LAB ASSIGNMENTS



Department of Electrical and Instrumentation Engineering

Submitted to: Dr. Mukesh Dalal
Submitted by: Madhav Jain-1024040079
Session:2026-2027

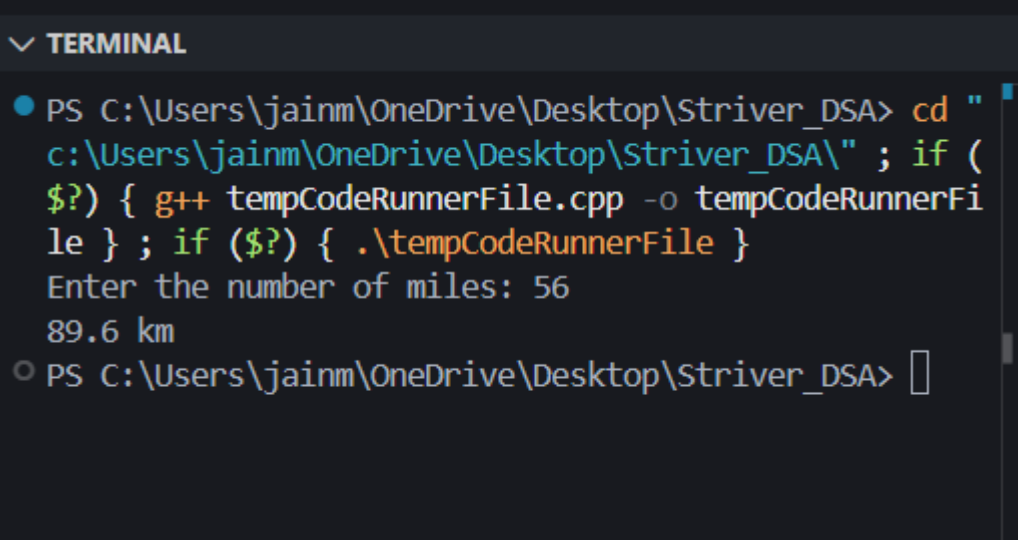
INDEX

S.No.	Title of Assignment
1.	Assignment 1: Basics
2.	Assignment 2: ARRAYS AND POINTERS
3.	Assignment 3: String Processing Operations
4.	Assignment 4: Linked List
5.	Assignment 5: Stack and its applications
6.	Assignment 6: Queues

Assignment 1

Q1: Write a program in C to convert miles into kilometers (Km). Hint: 1 Mile=1.609 Km.
[Use macros, relevant name and types for variables].

```
#include<iostream>
using namespace std;
#define miles_to_kms 1.6
int main(){
    double miles, kms;
    cout<<"Enter the number of miles: ";
    cin>>miles;
    kms=miles*miles_to_kms;
    cout<<kms<<" km";
    return 0;
}
```



A screenshot of a terminal window with a dark background. The title bar at the top says '✓ TERMINAL'. The terminal shows a command prompt where a user has navigated to 'C:\Users\jainm\OneDrive\Desktop\Striver_DSA' and run a C++ program. The program prompts 'Enter the number of miles: 56' and outputs '89.6 km'. The prompt then returns to 'PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA>'.

Q2: Write a program to find the number of positive, negative and zeros in a sequence of inputs (numbers) entered as data.

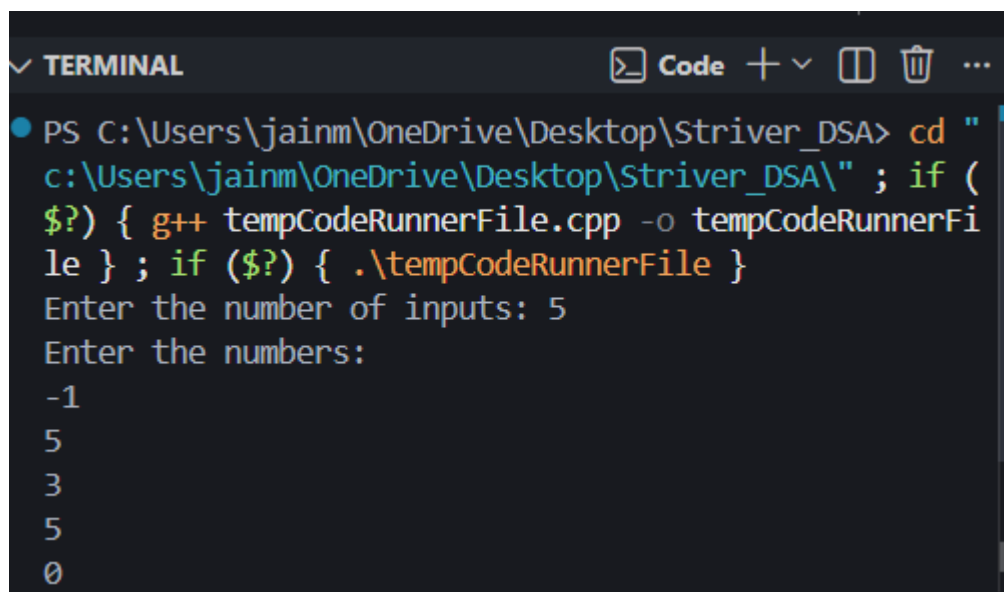
```
#include <iostream>
using namespace std;

int main() {
    int n,num;
    int positive=0,negative=0,zero=0;
    cout<<"Enter the number of inputs: ";
    cin>>n;
    cout<<"Enter the numbers:"<<endl;
    for(int i=0; i<n; i++){
        cin>>num;
```

```

    if(num>0)
        positive++;
    else if (num<0)
        negative++;
    else
        zero++;
}
cout<<"Positive numbers: " << positive<<endl;
cout<<"Negative numbers: " << negative<<endl;
cout<<"Zeros: " << zero<<endl;
return 0;
}

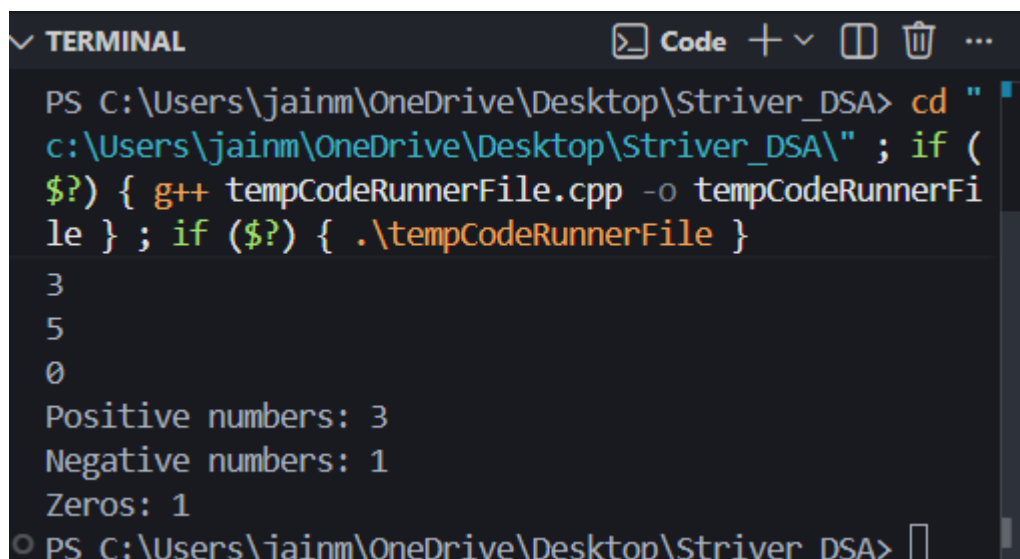
```



```

✓ TERMINAL
PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> cd "
c:\Users\jainm\OneDrive\Desktop\Striver_DSA\" ; if (
$?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFi
le } ; if ($?) { .\tempCodeRunnerFile }
Enter the number of inputs: 5
Enter the numbers:
-1
5
3
5
0

```



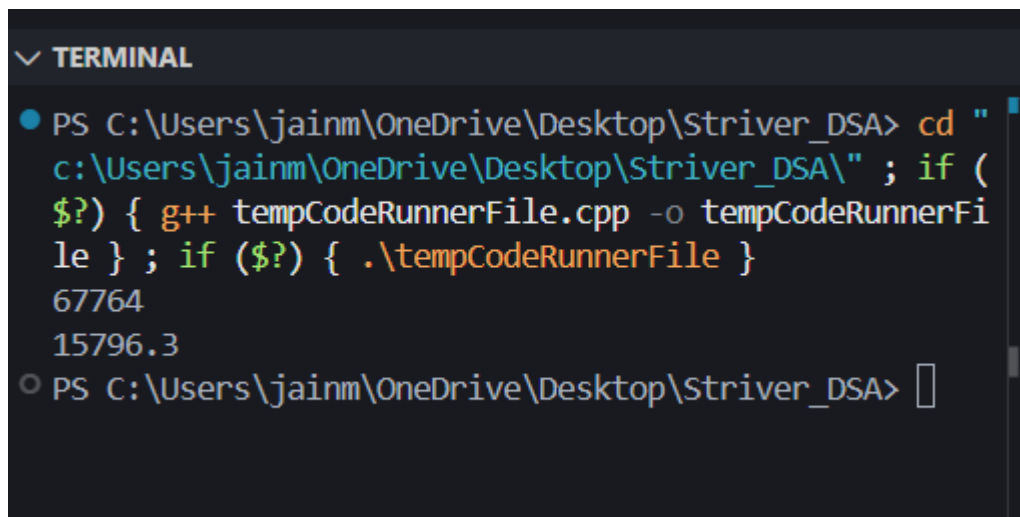
```

✓ TERMINAL
PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> cd "
c:\Users\jainm\OneDrive\Desktop\Striver_DSA\" ; if (
$?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFi
le } ; if ($?) { .\tempCodeRunnerFile }
3
5
0
Positive numbers: 3
Negative numbers: 1
Zeros: 1
PS C:\Users\jainm\OneDrive\Desktop\Striver DSA>

```

Q3: Compute the tax due based on a tax table given below: Program Input: Salary amount.
Program Output: Returns the tax due for $0.0 \leq \text{salary} \leq 150,000.00$; returns -1.0 if salary is outside the table range.

```
#include<iostream>
using namespace std;
int main(){
double salary,tax;
cin>>salary;
if(salary<0||salary>150000)
tax=-1.0;
else if(salary<=14999.99)
tax=salary*0.15;
else if(salary<=29999.99)
tax=2250+(salary-15000)*0.18;
else if(salary<=49999.99)
tax=5400+(salary-30000)*0.22;
else if(salary<=79999.99)
tax=11000+(salary-50000)*0.27;
else
tax=21600+(salary-80000)*0.33;
cout<<tax;
return 0;
}
```



```
✓ TERMINAL
● PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> cd "
c:\Users\jainm\OneDrive\Desktop\Striver_DSA\" ; if (
$?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFi
le } ; if ($?) { .\tempCodeRunnerFile }
67764
15796.3
○ PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> 
```

Q4: Write an interactive program (menu driven) in 'C' (using functions) to compute the area of a selected geometrical figure from a list of such figures (square, rectangle, and circle).

```
#include<iostream>
using namespace std;
```

```
double square(double side){
return side*side;
}
double rectangle(double length,double width){
return length*width;
}
double circle(double radius){
return 3.14159*radius*radius;
}
```

```
int main(){
int choice;
double a,b;
cout<<"1. Square\n2. Rectangle\n3. Circle\n";
cout<<"Enter your choice: ";
cin>>choice;
```

```
switch(choice){
case 1:
cout<<"Enter side: ";
cin>>a;
cout<<"Area = "<<square(a);
break;
case 2:
cout<<"Enter length and width: ";
cin>>a>>b;
cout<<"Area = "<<rectangle(a,b);
break;
case 3:
cout<<"Enter radius: ";
cin>>a;
cout<<"Area = "<<circle(a);
break;
default:
cout<<"Invalid choice";
}
return 0;
}
```

```
✓ TERMINAL
● PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> cd "
c:\Users\jainm\OneDrive\Desktop\Striver_DSA\" ; if (
$?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFi
le } ; if ($?) { .\tempCodeRunnerFile }
1. Square
2. Rectangle
3. Circle
Enter your choice: 3
Enter radius: 5
Area = 78.5397
○ PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> 
```

Q5: Write a program to display first n elements of Fibonacci series.

```
#include<iostream>
using namespace std;
int main(){
int n,a=0,b=1,c;
cin>>n;
for(int i=0;i<n;i++){
cout<<a<<" ";
c=a+b;
a=b;
b=c;
}
return 0;
}
```

```
✓ TERMINAL
● PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> cd "
c:\Users\jainm\OneDrive\Desktop\Striver_DSA\" ; if (
$?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFi
le } ; if ($?) { .\tempCodeRunnerFile }
5
0 1 1 2 3
○ PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> 
```

Q6: Write a program to print a table book from Table X to Table Y. X and Y are user inputs.

```
#include <stdio.h>

int main()
{
    int X, Y, i, j;

    printf("Enter X: ");
    scanf("%d", &X);

    printf("Enter Y: ");
    scanf("%d", &Y);

    for (i = X; i <= Y; i++)
    {
        printf("\nTable of %d:\n", i);

        for (j = 1; j <= 10; j++)
        {
            printf("%d x %d = %d\n", i, j, i * j);
        }
    }

    return 0;
}
```

Terminal

Code + ▾ □ 🗑️ ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { gcc tempCodeRunnerFile.c -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

Enter X: 2

Enter Y: 4

Table of 2:

2 x 1 = 2

2 x 2 = 4

2 x 3 = 6

2 x 4 = 8

2 x 5 = 10

2 x 6 = 12

2 x 7 = 14

2 x 8 = 16

2 x 9 = 18

2 x 10 = 20

Table of 3:

3 x 1 = 3

3 x 2 = 6

3 x 3 = 9

3 x 4 = 12

3 x 5 = 15

3 x 6 = 18

3 x 7 = 21

3 x 8 = 24

3 x 9 = 27

3 x 10 = 30

Terminal

Code + ▾ □ □ ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { gcc tempCodeRunnerFile.c -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

```
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
2 x 10 = 20
```

Table of 3:

```
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
3 x 10 = 30
```

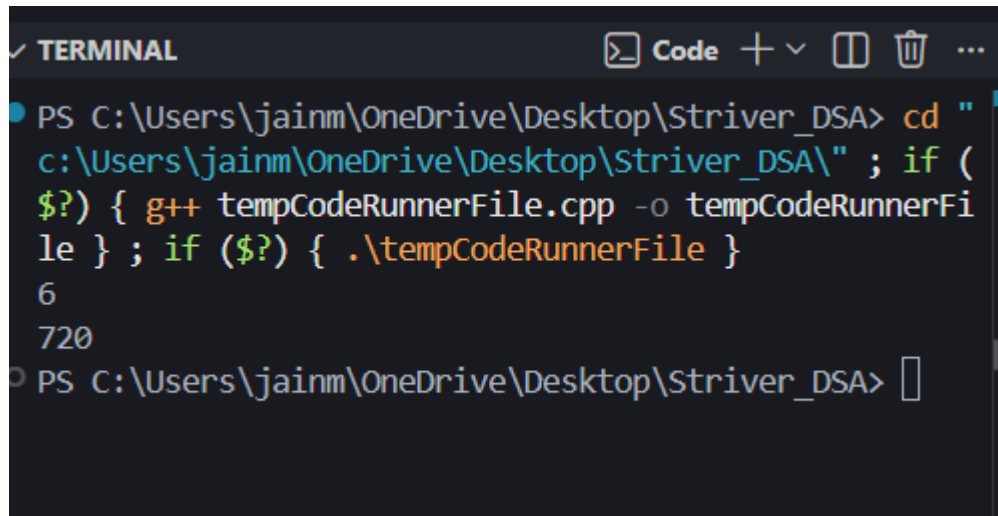
Table of 4:

```
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
4 x 4 = 16
4 x 5 = 20
4 x 6 = 24
4 x 7 = 28
4 x 8 = 32
4 x 9 = 36
4 x 10 = 40
```

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> 
```

Q7: Write a program to compute factorial of a number using iterative approach.

```
#include<iostream>
using namespace std;
int main(){
int n,fact=1;
cin>>n;
for(int i=1;i<=n;i++)
fact*=i;
cout<<fact;
return 0;
}
```

A screenshot of a terminal window with a dark background. The title bar says 'TERMINAL'. The command prompt shows the user is in the directory 'C:\Users\jainm\OneDrive\Desktop\Striver_DSA'. They run a command to compile and run a C++ program: 'cd "c:\Users\jainm\OneDrive\Desktop\Striver_DSA\" ; if (\$?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if (\$?) { .\tempCodeRunnerFile }'. The program outputs the number '6' on the first line and '720' on the second line. The prompt then returns to the command line.

```
✓ TERMINAL Code + - [ ] [ ] ...
PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> cd "
c:\Users\jainm\OneDrive\Desktop\Striver_DSA\" ; if (
$?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFi
le } ; if ($?) { .\tempCodeRunnerFile }
6
720
PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> [ ]
```

Q8: Write a program to swap two numbers using functions.

```
#include<iostream>
using namespace std;

void swap(int &a,int &b){
int temp=a;
a=b;
b=temp;
}
```

```
int main(){
int a,b;
cin>>a>>b;
swap(a,b);
cout<<a<<" "<<b;
return 0;
}
```

```
✓ TERMINAL
● PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> cd "
c:\Users\jainm\OneDrive\Desktop\Striver_DSA\" ; if (
$?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFi
le } ; if ($?) { .\tempCodeRunnerFile }
5
6
6 5
○ PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> █
```

Q9: Write a function that returns the first integer between n_min and n_max entered as data to the calling function (main).

```
#include<iostream>
using namespace std;
```

```
int findNumber(int n_min,int n_max){
int n;
while(cin>>n){
if(n>=n_min&& n<=n_max)
return n;
}
return -1;
}
```

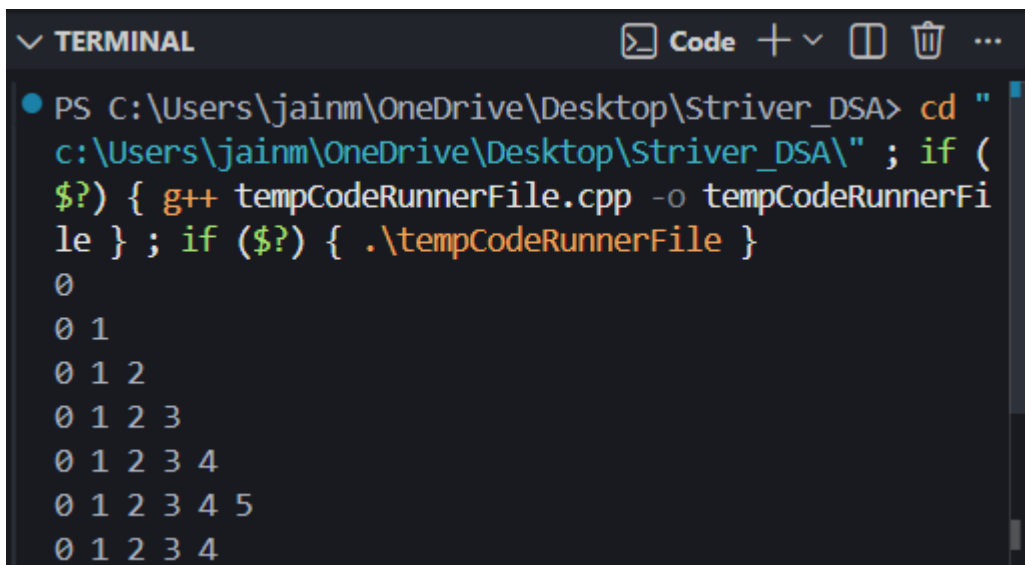
```
int main(){
int n_min,n_max,result;
cin>>n_min>>n_max;
result=findNumber(n_min,n_max);
cout<<result;
return 0;
}
```

```
✓ TERMINAL Code + ▾ 🗑️ ...
● PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> cd "
c:\Users\jainm\OneDrive\Desktop\Striver_DSA\" ; if (
$?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFi
le } ; if ($?) { .\tempCodeRunnerFile }
67
646
87
87
○ PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> 
```

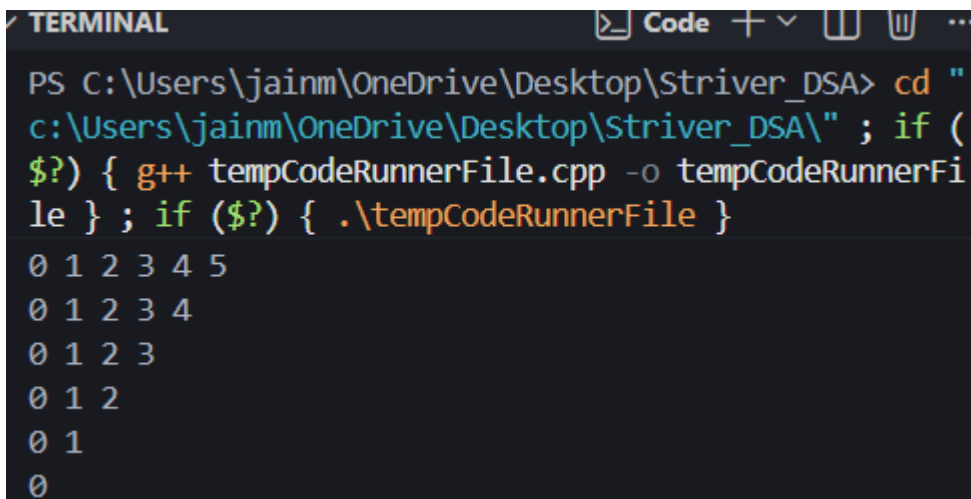
Q10: Write nests of loops that cause the following output to be displayed.

```
#include<iostream>
using namespace std;
```

```
int main(){
for(int i=0;i<=5;i++){
for(int j=0;j<=i;j++)
cout<<j<<" ";
cout<<endl;
}
for(int i=4;i>=0;i--){
for(int j=0;j<=i;j++)
cout<<j<<" ";
cout<<endl;
}
return 0;
}
```



```
▼ TERMINAL Code + ▢ ▢ ▢ ...
PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> cd "
c:\Users\jainm\OneDrive\Desktop\Striver_DSA\" ; if (
$?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFi
le } ; if ($?) { .\tempCodeRunnerFile }
0
0 1
0 1 2
0 1 2 3
0 1 2 3 4
0 1 2 3 4 5
0 1 2 3 4
```



```
▼ TERMINAL Code + ▢ ▢ ▢ ...
PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> cd "
c:\Users\jainm\OneDrive\Desktop\Striver_DSA\" ; if (
$?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFi
le } ; if ($?) { .\tempCodeRunnerFile }
0 1 2 3 4 5
0 1 2 3 4
0 1 2 3
0 1 2
0 1
0
```

Assignment 2

Q1. Write a program to check whether a given number is present in an array or not (Linear search).

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    int arr[100];
    for (int i = 0; i < n; i++) {
        cout << "Enter element " << i + 1 << ": ";
        cin >> arr[i];
    }

    int key;
    cout << "Enter number to search: ";
    cin >> key;

    int found = 0;
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            found = 1;
            cout << "Number found at position " << i + 1 << endl;
            break;
        }
    }

    if (found == 0) {
        cout << "Number not found in array" << endl;
    }

    return 0;
}
```



The screenshot shows a Windows Command Prompt (Terminal) window with the following text:

```
PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> cd "c:\Users\jainm\OneDrive\Desktop\Striver_DSA\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
Enter element 3: 54
Enter element 4: 34
Enter element 5: 78
Enter number to search: 54
Number found at position 3
PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> cd "c:\Users\jainm\OneDrive\Desktop\Striver_DSA\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

Q2: Write a program to get second maximum and second minimum elements in an array.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int n;
```

```
    cout << "Enter number of elements: ";
```

```
    cin >> n;
```

```
    int arr[100];
```

```
    for (int i = 0; i < n; i++) {
```

```
        cout << "Enter element " << i + 1 << ": ";
```

```
        cin >> arr[i];
```

```
    }
```

```
    int max1 = -1000000, max2 = -1000000;
```

```
    int min1 = 1000000, min2 = 1000000;
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (arr[i] > max1) {
```

```
            max2 = max1;
```

```
            max1 = arr[i];
```

```
        } else if (arr[i] > max2 && arr[i] != max1) {
```

```
            max2 = arr[i];
```

```
        }
```

```
        if (arr[i] < min1) {
```

```
            min2 = min1;
```

```
            min1 = arr[i];
```

```
        } else if (arr[i] < min2 && arr[i] != min1) {
```

```
            min2 = arr[i];
```

```
        }
```

```
    }
```

```
    cout << "Second maximum element is: " << max2 << endl;
```

```
    cout << "Second minimum element is: " << min2 << endl;
```

```

    return 0;
}

```



```

Terminal
PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> cd "c:\Users\jainm\OneDrive\Desktop\Striver_DSA" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
Enter element 1: 34
Enter element 2: 23
Enter element 3: 56
Enter element 4: 67
Second maximum element is: 56
Second minimum element is: 34
PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA>

```

Q3. Write a program to perform insertion (any location), deletion (any location) and traversal in an array.

```

#include <iostream>
using namespace std;

```

```

int main() {
    int arr[100];
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    for (int i = 0; i < n; i++) {
        cout << "Enter element " << i + 1 << ": ";
        cin >> arr[i];
    }

    int choice;
    do {
        cout << "\n1. Insert\n2. Delete\n3. Traverse\n4. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1) {
            int pos, value;
            cout << "Enter position to insert (1 to " << n + 1 << "): ";
            cin >> pos;
            cout << "Enter value to insert: ";
            cin >> value;

            for (int i = n; i >= pos; i--) {
                arr[i] = arr[i - 1];
            }
            arr[pos - 1] = value;

```

```

        n++;
        cout << "Element inserted successfully" << endl;
    }
    else if (choice == 2) {
        int pos;
        cout << "Enter position to delete (1 to " << n << "): ";
        cin >> pos;

        for (int i = pos - 1; i < n - 1; i++) {
            arr[i] = arr[i + 1];
        }
        n--;
        cout << "Element deleted successfully" << endl;
    }
    else if (choice == 3) {
        cout << "Array elements: ";
        for (int i = 0; i < n; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
    else if (choice == 4) {
        cout << "Exiting program" << endl;
    }
    else {
        cout << "Invalid choice" << endl;
    }
} while (choice != 4);

return 0;
}

```

```
Terminal
PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> cd "c:\Users\jainm\OneDrive\Desktop\Striver_DSA\" ; if ($?) { g++ tempCodeRunnerFile.cp
p -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
Enter number of elements: 4
Enter element 1: 45
Enter element 2: 23
Enter element 3:
78
Enter element 4: 87

Terminal
PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> cd "c:\Users\jainm\OneDrive\Desktop\Striver_DSA\" ; if ($?) { g++ tempCodeRunnerFile.cp
p -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
Enter element 4: 87

1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: 2

Terminal
PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> cd "c:\Users\jainm\OneDrive\Desktop\Striver_DSA\" ; if ($?) { g++ tempCodeRunnerFile.cp
p -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
2. Delete
3. Traverse
4. Exit
Enter your choice: 2
Enter position to delete (1 to 4): 3
Element deleted successfully
```

Q4. Write a menu driven program to perform addition, multiplication and subtraction of 2 arrays.

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter size of arrays: ";
    cin >> n;

    int a[100], b[100], result[100];
```

```

cout << "Enter elements of first array:" << endl;
for (int i = 0; i < n; i++) {
    cin >> a[i];
}

cout << "Enter elements of second array:" << endl;
for (int i = 0; i < n; i++) {
    cin >> b[i];
}

int choice;
cout << "\n1. Addition\n2. Multiplication\n3. Subtraction\n";
cout << "Enter your choice: ";
cin >> choice;

if (choice == 1) {
    for (int i = 0; i < n; i++) {
        result[i] = a[i] + b[i];
    }
    cout << "Result of addition: ";
}
else if (choice == 2) {
    for (int i = 0; i < n; i++) {
        result[i] = a[i] * b[i];
    }
    cout << "Result of multiplication: ";
}
else if (choice == 3) {
    for (int i = 0; i < n; i++) {
        result[i] = a[i] - b[i];
    }
    cout << "Result of subtraction: ";
}
else {
    cout << "Invalid choice" << endl;
    return 0;
}

for (int i = 0; i < n; i++) {
    cout << result[i] << " ";
}
cout << endl;

return 0;

```

}

```
Terminal
PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> cd "c:\Users\jainm\OneDrive\Desktop\Striver_DSA\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
Enter size of arrays: 4
Enter elements of first array:
23
54
67
78
Enter elements of second array:
```

Q5. Write a program to perform sorting while merging (Merge two sorted arrays into one sorted array).

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
    int n1, n2;
    cout << "Enter size of first sorted array: ";
    cin >> n1;

    int a[100];
    cout << "Enter elements in sorted order:" << endl;
    for (int i = 0; i < n1; i++) {
        cin >> a[i];
    }

    cout << "Enter size of second sorted array: ";
    cin >> n2;

    int b[100];
    cout << "Enter elements in sorted order:" << endl;
    for (int i = 0; i < n2; i++) {
        cin >> b[i];
    }

    int merged[200];
    int i = 0, j = 0, k = 0;

    while (i < n1 && j < n2) {
        if (a[i] <= b[j]) {
            merged[k] = a[i];
            i++;
        } else {
            merged[k] = b[j];
            j++;
        }
    }
}
```

```

    }
    k++;
}

while (i < n1) {
    merged[k] = a[i];
    i++;
    k++;
}

while (j < n2) {
    merged[k] = b[j];
    j++;
    k++;
}

cout << "Merged sorted array: ";
for (int x = 0; x < k; x++) {
    cout << merged[x] << " ";
}
cout << endl;

return 0;
}

```



```

Terminal
PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> cd "c:\Users\jainm\OneDrive\Desktop\Striver_DSA\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
Enter elements in sorted order:
1 2 3 4
Enter size of second sorted array: 3
Enter elements in sorted order:
4 5 6
Merged sorted array: 1 2 3 4 4 5 6
PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA>

```

Q6. Write the above programs (1,2, and 3) using functions and call by address only.

```
#include <iostream>
using namespace std;

void linearSearch(int *arr, int n, int key) {
    int found = 0;
    for (int i = 0; i < n; i++) {
        if (*(arr + i) == key) {
            found = 1;
            cout << "Number found at position " << i + 1 << endl;
            break;
        }
    }
    if (found == 0) {
        cout << "Number not found in array" << endl;
    }
}

void secondMaxMin(int *arr, int n) {
    int max1 = -1000000, max2 = -1000000;
    int min1 = 1000000, min2 = 1000000;

    for (int i = 0; i < n; i++) {
        if (*(arr + i) > max1) {
            max2 = max1;
            max1 = *(arr + i);
        } else if (*(arr + i) > max2 && *(arr + i) != max1) {
            max2 = *(arr + i);
        }

        if (*(arr + i) < min1) {
            min2 = min1;
            min1 = *(arr + i);
        } else if (*(arr + i) < min2 && *(arr + i) != min1) {
            min2 = *(arr + i);
        }
    }

    cout << "Second maximum element is: " << max2 << endl;
    cout << "Second minimum element is: " << min2 << endl;
}
```

```
}
```

```
void insertElement(int *arr, int *n, int pos, int value) {  
    for (int i = *n; i >= pos; i--) {  
        *(arr + i) = *(arr + i - 1);  
    }  
    *(arr + pos - 1) = value;  
    *n = *n + 1;  
    cout << "Element inserted successfully" << endl;  
}
```

```
void deleteElement(int *arr, int *n, int pos) {  
    for (int i = pos - 1; i < *n - 1; i++) {  
        *(arr + i) = *(arr + i + 1);  
    }  
    *n = *n - 1;  
    cout << "Element deleted successfully" << endl;  
}
```

```
void traverseArray(int *arr, int n) {  
    cout << "Array elements: ";  
    for (int i = 0; i < n; i++) {  
        cout << *(arr + i) << " ";  
    }  
    cout << endl;  
}
```

```
int main() {  
    int arr[100];  
    int n;  
    cout << "Enter number of elements: ";  
    cin >> n;  
  
    for (int i = 0; i < n; i++) {  
        cout << "Enter element " << i + 1 << ": ";  
        cin >> arr[i];  
    }
```

```
    int key;  
    cout << "Enter number to search: ";  
    cin >> key;  
    linearSearch(arr, n, key);  
  
    secondMaxMin(arr, n);
```

```

int choice;
do {
    cout << "\n1. Insert\n2. Delete\n3. Traverse\n4. Exit\n";
    cout << "Enter your choice: ";
    cin >> choice;

    if (choice == 1) {
        int pos, value;
        cout << "Enter position to insert (1 to " << n + 1 << "): ";
        cin >> pos;
        cout << "Enter value to insert: ";
        cin >> value;
        insertElement(arr, &n, pos, value);
    }
    else if (choice == 2) {
        int pos;
        cout << "Enter position to delete (1 to " << n << "): ";
        cin >> pos;
        deleteElement(arr, &n, pos);
    }
    else if (choice == 3) {
        traverseArray(arr, n);
    }
    else if (choice == 4) {
        cout << "Exiting program" << endl;
    }
    else {
        cout << "Invalid choice" << endl;
    }

} while (choice != 4);

return 0;
}

```



```

Terminal
PS C:\Users\jainm\OneDrive\Desktop\Striver_DSA> cd "c:\Users\jainm\OneDrive\Desktop\Striver_DSA\" ; if ($?) { g++ tempCodeRunnerFile.cpp
p -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
Enter number of elements: 4
Enter element 1: 2
Enter element 2: 4
Enter element 3: 7
Enter element 4: 8
Enter number to search: 5
Number not found in array

```

Assignment 3

Q1: Write a program to implement strlen() function.

Code: Without Inbuilt function:

```
#include <iostream>
using namespace std;
int main(){
    char str[100];
    int length=0;
    cout<<"enter the string: "<<endl;
    cin.getline(str,100);
    while (str[length]!='\0'){
        length++;
    }
    cout<<"length= "<<length;
    return 0;
}
```

Output:



```
Terminal
Code + - [ ] [ ] ...
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ strlen.cpp -o strlen } ; if ($?) { .\strlen }
enter the string:
madhav jain
length= 11
PS C:\Users\jainm\OneDrive\Desktop\dsa lab>
```

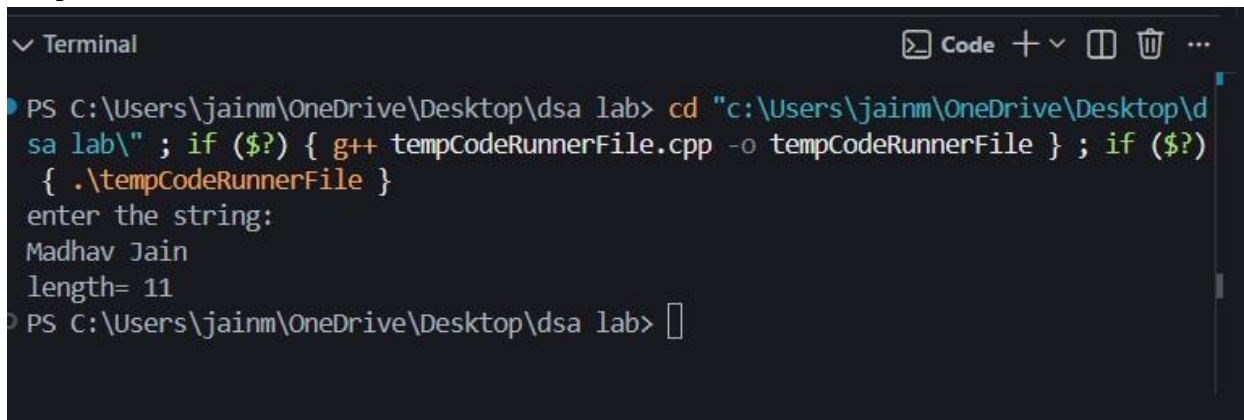
Code: With inbuilt function:

```
#include <iostream>
#include <cstring>
using namespace std;
int main(){
    char str[100];
    cout<<"enter the string: "<<endl;
    cin.getline(str,100);

    cout<<"length= "<<strlen(str);
}
```

```
    return 0;
}
```

Output: With inbuilt function:



```
Terminal
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
enter the string:
Madhav Jain
length= 11
PS C:\Users\jainm\OneDrive\Desktop\dsa lab>
```

Q2: Write a program to implement strcpy() function.

Code: Without Inbuilt Function

```
#include <iostream>
using namespace std;
int main(){
    char alpha[100];
    char beta[100];
    cout<<"enter string alpha"<<endl;
    cin.getline(alpha,100);
    int i=0;
    while(alpha[i]!='\0'){
        beta[i]=alpha[i];
        i++;
    }
    beta[i]='\0';
    cout<<"copied string: "<<beta;

    return 0;
}
```

Output: Without Inbuilt function:

```
Terminal
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ strcpy.cpp -o strcpy } ; if ($?) { .\strcpy }
enter string alpha
Madhav Jain
copied string: Madhav Jain
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> 
```

Code: With Inbuilt function:

```
#include <iostream>
#include <cstring>
using namespace std;
int main(){
    char alpha[100];
    char beta[100];
    cout<<"enter string alpha"<<endl;
    cin.getline(alpha,100);
    strcpy(beta,alpha);
    cout<<beta;

    return 0;
}
```

Output: With Inbuilt function:

```
Terminal
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
enter string alpha
Madhav Jain
Madhav Jain
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> 
```

Q3: Write a program to implement strcat() function.

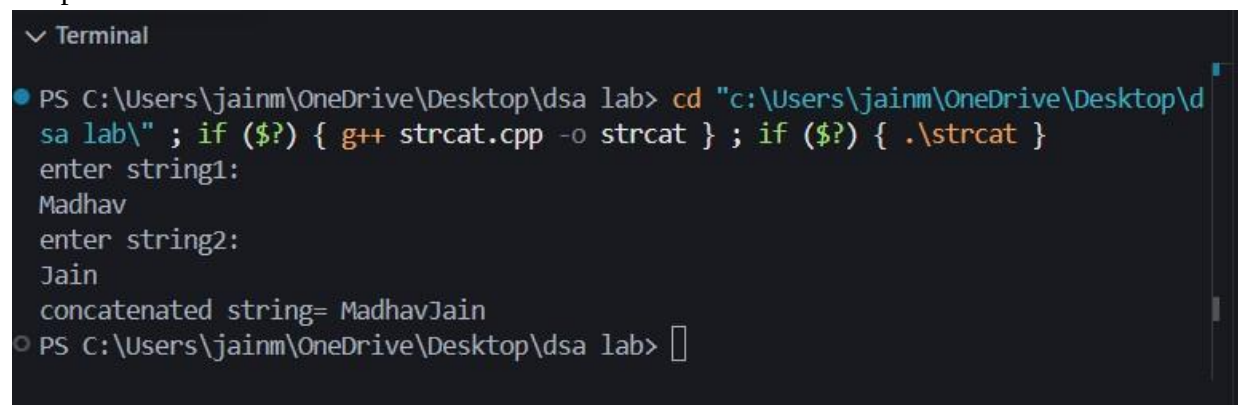
Code: Without Inbuilt function

```
#include <iostream>
using namespace std;
int main(){
    char str1[100];
    char str2[100];
    cout<<"enter string1: "<<endl;
    cin.getline(str1,100);

    cout<<"enter string2: "<<endl;
    cin.getline(str2,100);
    int i=0;
    while(str1[i] !='\0'){
        i++;
    }
    int j=0;
    while(str2[j] !='\0'){
        str1[i]=str2[j];
        i++;
        j++;
    }
    str1[i]='\0';
    cout<<"concatenated string= "<<str1;

    return 0;
}
```

Output: Without Inbuilt function:

A screenshot of a Windows terminal window with a dark background. The title bar says "Terminal". The command prompt shows the user navigating to the directory "C:\Users\jainm\OneDrive\Desktop\dsa lab\" and running a C++ program named "strcat.cpp". The program prompts for two strings: "enter string1:" and "enter string2:". The user enters "Madhav" and "Jain" respectively. The program then outputs "concatenated string= MadhavJain". The prompt returns to the command line.

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ strcat.cpp -o strcat } ; if ($?) { .\strcat }
enter string1:
Madhav
enter string2:
Jain
concatenated string= MadhavJain
PS C:\Users\jainm\OneDrive\Desktop\dsa lab>
```

Code: With Inbuilt function:

```
#include <iostream>
#include <cstring>
using namespace std;
int main(){
    char str1[100];
    char str2[100];
    cout<<"enter string1: "<<endl;
    cin.getline(str1,100);

    cout<<"enter string2: "<<endl;
    cin.getline(str2,100);
    strcat(str1,str2);
    cout<<str1;

    return 0;
}
```

Output: With inbuilt function:

```
Terminal
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
enter string1:
Madhav
enter string2:
Jain
MadhavJain
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> 
```

Q4: Write a program to implement strcmp() function.

Code: Without Inbuilt function:

```
#include <iostream>
using namespace std;
int main(){
    char str1[100];
    char str2[100];

    cout<<"enter string1: "<<endl;
    cin.getline(str1,100);

    cout<<"enter string2: "<<endl;
    cin.getline(str2,100);

    int i=0;
    while(str1[i]==str2[i] && str1[i]!='\0'){
        i++;
    }
    if(str1[i]==str2[i]){
        cout<<"equal strings"<<endl;
    }
    else if(str1[i]>str2[i]){
        cout<<"string 1 is greater"<<endl;
    }
    else{
        cout<<"String 2 is greater"<<endl;
    }

    return 0;
}
```

Output: Without Inbuilt function

```
Terminal
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ strcmp.cpp -o strcmp } ; if ($?) { .\strcmp }
enter string1:
Madhav
enter string2:
Jain
string 1 is greater
PS C:\Users\jainm\OneDrive\Desktop\dsa lab>
```

Code: With Inbuilt function:

```
#include <iostream>
#include <cstring>
using namespace std;
int main(){
    char str1[100];
    char str2[100];

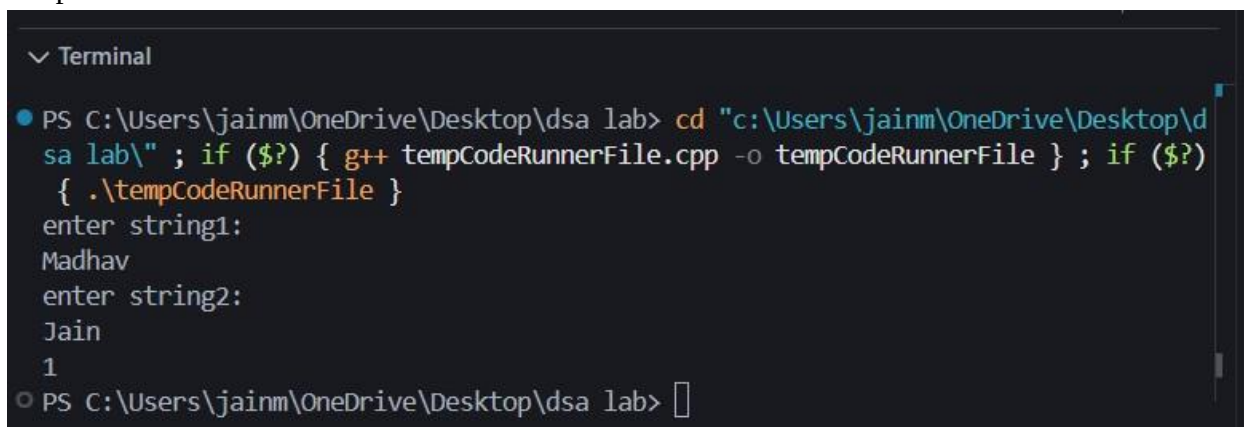
    cout<<"enter string1: "<<endl;
    cin.getline(str1,100);

    cout<<"enter string2: "<<endl;
    cin.getline(str2,100);

    int result=strcmp(str1,str2);
    cout<<result<<endl;

    return 0;
}
```

Output: With Inbuilt function



```
Terminal
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
enter string1:
Madhav
enter string2:
Jain
1
PS C:\Users\jainm\OneDrive\Desktop\dsa lab>
```

Q5: WAP to demonstrate limitations of Two-Dimensional Array of Characters.

Code:

```
#include <iostream>
using namespace std;
int main(){
    char menu[3][20];
    cout<<"Enter the name of 3 dishes:\n";
    for(int i=0;i<3;i++){
        cin.getline(menu[i],20);
    }
    cout<<"Dishes ordered are: "<<endl;
    for(int i=0;i<3;i++){
        cout<<menu[i]<<endl;
    }

    return 0;
}
```

Output:

```
Terminal
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ chararray.cpp -o chararray } ; if ($?) { .\chararray }
Enter the name of 3 dishes:
Dosa
Chana Bhatura
Soup
Dishes ordered are:
Dosa
Chana Bhatura

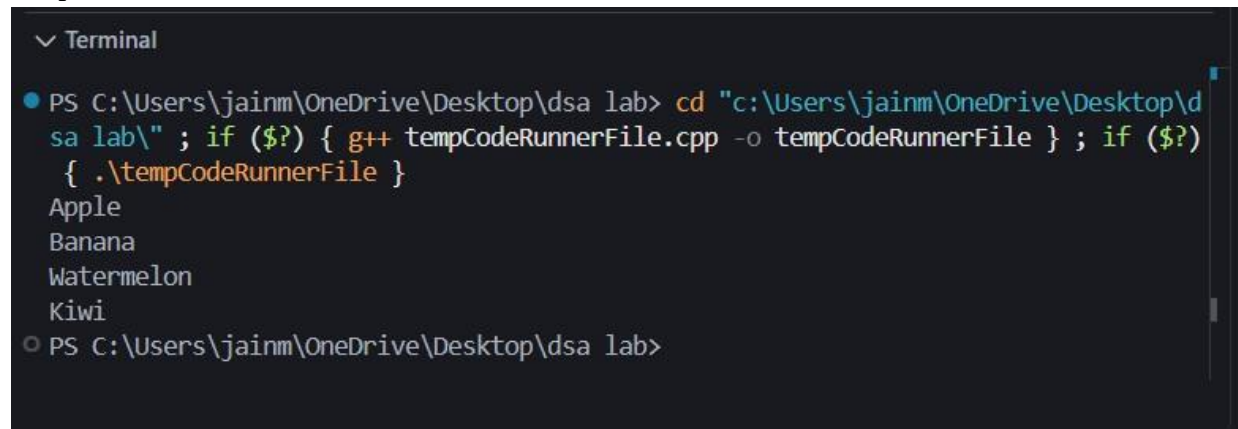
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ chararray.cpp -o chararray } ; if ($?) { .\chararray }
Chana Bhatura
Soup
Dishes ordered are:
Dosa
Chana Bhatura
Soup
PS C:\Users\jainm\OneDrive\Desktop\dsa lab>
```

Q6: WAP to demonstrate an array of Pointers to Strings.

Code:

```
#include <iostream>
using namespace std;
int main() {
    const char* names[] = {
        "Apple",
        "Banana",
        "Watermelon",
        "Kiwi"
    };
    for (int i = 0; i < 4; i++) {
        cout << names[i] << endl;
    }
    return 0;
}
```

Output:



```
Terminal
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
Apple
Banana
Watermelon
Kiwi
PS C:\Users\jainm\OneDrive\Desktop\dsa lab>
```

ASSIGNMENT 4

Q1. Write a program to implement single Link List (with function insertion at first node, insertion at last node, deletion at first node, deletion at last node, insertion and deletion at middle node and traversing of link list).

```
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *next;
};

Node *head = NULL;

void insertFirst()
{
    int x;

    cout << "Enter value: ";
    cin >> x;

    Node *newNode = new Node;

    newNode->data = x;
    newNode->next = head;

    head = newNode;

    cout << "Node inserted at first\n";
}

void insertLast()
{
    int x;

    cout << "Enter value: ";
    cin >> x;

    Node *newNode = new Node;

    newNode->data = x;
```

```

newNode->next = NULL;

if (head == NULL)
{
    head = newNode;
}
else
{
    Node *temp = head;

    while (temp->next != NULL)
    {
        temp = temp->next;
    }

    temp->next = newNode;
}

cout << "Node inserted at last\n";
}

void deleteFirst()
{
    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }

    Node *temp = head;

    head = head->next;

    delete temp;

    cout << "First node deleted\n";
}

void deleteLast()
{
    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }

```

```

    }

    if (head->next == NULL)
    {
        delete head;
        head = NULL;
        cout << "Last node deleted\n";
        return;
    }

    Node *temp = head;

    while (temp->next->next != NULL)
    {
        temp = temp->next;
    }

    delete temp->next;
    temp->next = NULL;

    cout << "Last node deleted\n";
}

void insertMiddle()
{
    int x, pos;

    cout << "Enter value: ";
    cin >> x;

    cout << "Enter position: ";
    cin >> pos;

    Node *newNode = new Node;

    newNode->data = x;

    if (pos == 1)
    {
        newNode->next = head;
        head = newNode;
        return;
    }

```

```

Node *temp = head;

for (int i = 1; i < pos - 1; i++)
{
    if (temp == NULL)
    {
        cout << "Invalid position\n";
        delete newNode;
        return;
    }

    temp = temp->next;
}

if (temp == NULL)
{
    cout << "Invalid position\n";
    delete newNode;
    return;
}

newNode->next = temp->next;
temp->next = newNode;

cout << "Node inserted\n";
}

void deleteMiddle()
{
    int pos;

    cout << "Enter position: ";
    cin >> pos;

    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }

    if (pos == 1)
    {
        Node *temp = head;
        head = head->next;
    }
}

```

```

        delete temp;

        cout << "Node deleted\n";
        return;
    }

    Node *temp = head;

    for (int i = 1; i < pos - 1; i++)
    {
        if (temp == NULL)
        {
            cout << "Invalid position\n";
            return;
        }

        temp = temp->next;
    }

    if (temp == NULL || temp->next == NULL)
    {
        cout << "Invalid position\n";
        return;
    }

    Node *del = temp->next;

    temp->next = del->next;

    delete del;

    cout << "Node deleted\n";
}

void display()
{
    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }

    Node *temp = head;

```

```

    cout << "Linked List: ";

    while (temp != NULL)
    {
        cout << temp->data << " ";
        temp = temp->next;
    }

    cout << endl;
}

int main()
{
    int choice;

    do
    {
        cout << "\n MENU \n";
        cout << "1. Insert at First\n";
        cout << "2. Insert at Last\n";
        cout << "3. Delete at First\n";
        cout << "4. Delete at Last\n";
        cout << "5. Insert at Middle\n";
        cout << "6. Delete at Middle\n";
        cout << "7. Display / Traverse\n";
        cout << "8. Exit\n";

        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1:
                insertFirst();
                break;

            case 2:
                insertLast();
                break;

            case 3:
                deleteFirst();
                break;

```

```
case 4:
    deleteLast();
    break;

case 5:
    insertMiddle();
    break;

case 6:
    deleteMiddle();
    break;

case 7:
    display();
    break;

case 8:
    cout << "Program ended\n";
    break;

default:
    cout << "Wrong choice\n";
    }

} while (choice != 8);

return 0;
}
```

Terminal

Code + ▾ □ □ ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 1

Enter value: 10

Node inserted at first

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 2

Enter value: 20

Node inserted at last

Terminal

Code + ▾ □ 🗑 ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 2

Enter value: 30

Node inserted at last

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 7

Linked List: 10 20 30

Terminal

Code + ▾ □ 🗑 ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 5

Enter value: 15

Enter position: 2

Node inserted

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 6

Enter position: 3

Node deleted

Terminal

Code + ▾ □ 🗑 ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 7

Linked List: 10 15 30

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 3

First node deleted

Terminal

Code + ▾ 🗑️ ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 10

Wrong choice

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 7

Linked List: 15 30

Terminal

Code + ▾ □ 🗑️ ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 4

Last node deleted

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 7

Linked List: 15

MENU

Terminal

Code + ▾ 🗑️ ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

7. Display / Traverse

8. Exit

Enter your choice: 7

Linked List: 15

MENU

1. Insert at First

2. Insert at Last

3. Delete at First

4. Delete at Last

5. Insert at Middle

6. Delete at Middle

7. Display / Traverse

8. Exit

Enter your choice: 8

Program ended

PS C:\Users\jainm\OneDrive\Desktop\dsa lab> █

Q2. Write a program to implement Doubly Link List (with function insertion at first node, insertion at last node, deletion at first node, deletion at last node, insertion and deletion at middle node and traversing of link list).

```
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *prev;
    Node *next;
};

Node *head = NULL;

void insertFirst()
{
    int x;

    cout << "Enter value: ";
    cin >> x;

    Node *newNode = new Node;

    newNode->data = x;
    newNode->prev = NULL;
    newNode->next = head;

    if (head != NULL)
    {
        head->prev = newNode;
    }

    head = newNode;

    cout << "Node inserted at first\n";
}

void insertLast()
{
    int x;

    cout << "Enter value: ";
```

```

cin >> x;

Node *newNode = new Node;

newNode->data = x;
newNode->next = NULL;

if (head == NULL)
{
    newNode->prev = NULL;
    head = newNode;
}
else
{
    Node *temp = head;

    while (temp->next != NULL)
    {
        temp = temp->next;
    }

    temp->next = newNode;
    newNode->prev = temp;
}

cout << "Node inserted at last\n";
}

void deleteFirst()
{
    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }

    Node *temp = head;

    head = head->next;

    if (head != NULL)
    {
        head->prev = NULL;
    }
}

```

```

delete temp;

cout << "First node deleted\n";
}

void deleteLast()
{
    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }

    if (head->next == NULL)
    {
        delete head;
        head = NULL;

        cout << "Last node deleted\n";
        return;
    }

    Node *temp = head;

    while (temp->next != NULL)
    {
        temp = temp->next;
    }

    temp->prev->next = NULL;

    delete temp;

    cout << "Last node deleted\n";
}

void insertMiddle()
{
    int x, pos;

    cout << "Enter value: ";
    cin >> x;

```

```

cout << "Enter position: ";
cin >> pos;

Node *newNode = new Node;

newNode->data = x;

if (pos == 1)
{
    newNode->prev = NULL;
    newNode->next = head;

    if (head != NULL)
    {
        head->prev = newNode;
    }

    head = newNode;

    cout << "Node inserted\n";
    return;
}

Node *temp = head;

for (int i = 1; i < pos - 1; i++)
{
    if (temp == NULL)
    {
        cout << "Invalid position\n";
        delete newNode;
        return;
    }

    temp = temp->next;
}

if (temp == NULL)
{
    cout << "Invalid position\n";
    delete newNode;
    return;
}

```

```

newNode->next = temp->next;
newNode->prev = temp;

if (temp->next != NULL)
{
    temp->next->prev = newNode;
}

temp->next = newNode;

cout << "Node inserted\n";
}

void deleteMiddle()
{
    int pos;

    cout << "Enter position: ";
    cin >> pos;

    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }

    if (pos == 1)
    {
        Node *temp = head;

        head = head->next;

        if (head != NULL)
        {
            head->prev = NULL;
        }

        delete temp;

        cout << "Node deleted\n";
        return;
    }

    Node *temp = head;

```

```

for (int i = 1; i < pos; i++)
{
    if (temp == NULL)
    {
        cout << "Invalid position\n";
        return;
    }

    temp = temp->next;
}

if (temp == NULL)
{
    cout << "Invalid position\n";
    return;
}

if (temp->next != NULL)
{
    temp->next->prev = temp->prev;
}

if (temp->prev != NULL)
{
    temp->prev->next = temp->next;
}

delete temp;

cout << "Node deleted\n";
}

void display()
{
    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }

    Node *temp = head;

    cout << "Doubly Linked List: ";

```

```

while (temp != NULL)
{
    cout << temp->data << " ";
    temp = temp->next;
}

cout << endl;
}

int main()
{
    int choice;

    do
    {
        cout << "\nMENU \n";
        cout << "1. Insert at First\n";
        cout << "2. Insert at Last\n";
        cout << "3. Delete at First\n";
        cout << "4. Delete at Last\n";
        cout << "5. Insert at Middle\n";
        cout << "6. Delete at Middle\n";
        cout << "7. Display / Traverse\n";
        cout << "8. Exit\n";

        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1:
                insertFirst();
                break;

            case 2:
                insertLast();
                break;

            case 3:
                deleteFirst();
                break;

            case 4:

```

```
        deleteLast();
        break;

    case 5:
        insertMiddle();
        break;

    case 6:
        deleteMiddle();
        break;

    case 7:
        display();
        break;

    case 8:
        cout << "Program ended\n";
        break;

    default:
        cout << "Wrong choice\n";
    }

} while (choice != 8);

return 0;
}
```

Terminal

Code + - [] [] ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

```
1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit
Enter your choice: 1
Enter value: 10
Node inserted at first
```

MENU

```
1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit
Enter your choice: 2
Enter value: 20
Node inserted at last
```

Terminal

Code + ▾ 🗑️ ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 2

Enter value: 30

Node inserted at last

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 7

Doubly Linked List: 10 20 30

Terminal

Code + - [] [] ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 5

Enter value: 15

Enter position: 2

Node inserted

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 7

Doubly Linked List: 10 15 20 30

Terminal

Code + ▾ 📄 🗑️ ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 6

Enter position: 3

Node deleted

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 7

Doubly Linked List: 10 15 30

MENU

```
Terminal
Code + - [ ] [ ] ...

PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }

MENU
1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit
Enter your choice: 3
First node deleted

MENU
1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit
Enter your choice: 7
Doubly Linked List: 15 30
```

Terminal

Code + ▾ □ 🗑️ ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 4
Last node deleted

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 7
Doubly Linked List: 15

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 8
Program ended

Q3. Write a program to implement Circular Doubly Link List (with function insertion at first node, insertion at last node, deletion at first node, deletion at last node, insertion and deletion at middle node and traversing of link list).

```
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *prev;
    Node *next;
};

Node *head = NULL;

void insertFirst()
{
    int x;

    cout << "Enter value: ";
    cin >> x;

    Node *newNode = new Node;

    newNode->data = x;

    if (head == NULL)
    {
        newNode->next = newNode;
        newNode->prev = newNode;
        head = newNode;
    }
    else
    {
        Node *last = head->prev;

        newNode->next = head;
        newNode->prev = last;

        last->next = newNode;
        head->prev = newNode;

        head = newNode;
    }
}
```

```

    }

    cout << "Node inserted at first\n";
}

void insertLast()
{
    int x;

    cout << "Enter value: ";
    cin >> x;

    Node *newNode = new Node;

    newNode->data = x;

    if (head == NULL)
    {
        newNode->next = newNode;
        newNode->prev = newNode;
        head = newNode;
    }
    else
    {
        Node *last = head->prev;

        newNode->next = head;
        newNode->prev = last;

        last->next = newNode;
        head->prev = newNode;
    }

    cout << "Node inserted at last\n";
}

void deleteFirst()
{
    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }
}

```

```

if (head->next == head)
{
    delete head;
    head = NULL;
}
else
{
    Node *last = head->prev;
    Node *temp = head;

    head = head->next;

    head->prev = last;
    last->next = head;

    delete temp;
}

cout << "First node deleted\n";
}

void deleteLast()
{
    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }

    if (head->next == head)
    {
        delete head;
        head = NULL;
    }
    else
    {
        Node *last = head->prev;
        Node *secondLast = last->prev;

        secondLast->next = head;
        head->prev = secondLast;

        delete last;
    }
}

```

```
    cout << "Last node deleted\n";  
}
```

```
void insertMiddle()
```

```
{  
    int x, pos;  
  
    cout << "Enter value: ";  
    cin >> x;
```

```
    cout << "Enter position: ";  
    cin >> pos;
```

```
    Node *newNode = new Node;
```

```
    newNode->data = x;
```

```
    if (head == NULL)  
    {  
        cout << "List is empty\n";  
        delete newNode;  
        return;  
    }
```

```
    if (pos == 1)  
    {  
        Node *last = head->prev;
```

```
        newNode->next = head;  
        newNode->prev = last;
```

```
        last->next = newNode;  
        head->prev = newNode;
```

```
        head = newNode;
```

```
        cout << "Node inserted\n";  
        return;  
    }
```

```
    Node *temp = head;
```

```
    for (int i = 1; i < pos - 1; i++)
```

```

{
    temp = temp->next;

    if (temp == head)
    {
        cout << "Invalid position\n";
        delete newNode;
        return;
    }
}

newNode->next = temp->next;
newNode->prev = temp;

temp->next->prev = newNode;
temp->next = newNode;

cout << "Node inserted\n";
}

void deleteMiddle()
{
    int pos;

    cout << "Enter position: ";
    cin >> pos;

    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }

    if (pos == 1)
    {
        if (head->next == head)
        {
            delete head;
            head = NULL;
        }
        else
        {
            Node *last = head->prev;
            Node *temp = head;

```

```

        head = head->next;

        head->prev = last;
        last->next = head;

        delete temp;
    }

    cout << "Node deleted\n";
    return;
}

Node *temp = head;

for (int i = 1; i < pos; i++)
{
    temp = temp->next;

    if (temp == head)
    {
        cout << "Invalid position\n";
        return;
    }
}

temp->prev->next = temp->next;
temp->next->prev = temp->prev;

delete temp;

cout << "Node deleted\n";
}

void display()
{
    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }

    Node *temp = head;

```

```

cout << "Circular Doubly Linked List: ";

do
{
    cout << temp->data << " ";
    temp = temp->next;
}
while (temp != head);

cout << endl;
}

int main()
{
    int choice;

    do
    {
        cout << "\nMENU\n";
        cout << "1. Insert at First\n";
        cout << "2. Insert at Last\n";
        cout << "3. Delete at First\n";
        cout << "4. Delete at Last\n";
        cout << "5. Insert at Middle\n";
        cout << "6. Delete at Middle\n";
        cout << "7. Display / Traverse\n";
        cout << "8. Exit\n";

        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1:
                insertFirst();
                break;

            case 2:
                insertLast();
                break;

            case 3:
                deleteFirst();
                break;

```

```
    case 4:
        deleteLast();
        break;

    case 5:
        insertMiddle();
        break;

    case 6:
        deleteMiddle();
        break;

    case 7:
        display();
        break;

    case 8:
        cout << "Program ended\n";
        break;

    default:
        cout << "Wrong choice\n";
    }

} while (choice != 8);

return 0;
}
```

Terminal

Code + - [] [] ...

```
● PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 1

Enter value: 10

Node inserted at first

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 2

Enter value: 20

Node inserted at last

Terminal

Code + ▾ □ □ ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\d
```

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 2

Enter value: 30

Node inserted at last

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 7

Circular Doubly Linked List: 10 20 30

MENU

Code + View + ...

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Last node deleted

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Circular Doubly Linked List: 15

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Program ended

```
PS C:\Users\jainS\OneDrive\Desktop\dsa lab>
```

Terminal

Code + ▾ □ 🗑️ ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab"
MENU
```

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 5

Enter value: 15

Enter position: 2

Node inserted

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 7

Circular Doubly Linked List: 10 15 20 30

Assignment 5

Q1. Write a menu driven program with 4 options (Push, Pop, Display, and Exit) to demonstrate the working of stacks using arrays.

Code:

```
#include <iostream>
using namespace std;

#define MAX 5

int stack[MAX];
int top = -1;

void push()
{
    int x;

    if (top == MAX - 1)
    {
        cout << "Stack Overflow" << endl;
    }
    else
    {
        cout << "Enter value: ";
        cin >> x;

        top++;
        stack[top] = x;

        cout << "Value pushed successfully" << endl;
    }
}

void pop()
{
    if (top == -1)
    {
        cout << "Stack Underflow" << endl;
    }
    else
    {
        cout << "Popped value: " << stack[top] << endl;
        top--;
    }
}
```

```
void display()
{
    if (top == -1)
    {
        cout << "Stack is empty" << endl;
    }
    else
    {
        cout << "Stack elements are:" << endl;

        for (int i = top; i >= 0; i--)
        {
            cout << stack[i] << endl;
        }
    }
}
```

```
int main()
{
    int choice;

    while (true)
    {
        cout << "\n1. Push" << endl;
        cout << "2. Pop" << endl;
        cout << "3. Display" << endl;
        cout << "4. Exit" << endl;

        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            push();
        }
        else if (choice == 2)
        {
            pop();
        }
        else if (choice == 3)
        {
            display();
        }
    }
}
```

```

else if (choice == 4)
{
    cout << "Program ended" << endl;
    break;
}
else
{
    cout << "Invalid choice" << endl;
}
}

return 0;
}

```

Output:

```

Terminal
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1

```

```

Terminal
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
Enter your choice: 1
Enter value: 4
Value pushed successfully

1. Push
2. Pop

```

Terminal

Code + - [] [] ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 2

Terminal

Code + - [] [] ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

4. Exit

Enter your choice: 2

Popped value: 4

1. Push
2. Pop

Terminal

Code + - [] [] ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

1. Push

2. Pop
3. Display
4. Exit

Enter your choice: 3

Stack is empty

Terminal

Code + - [] [] ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 4

```
Terminal
Code + - [ ] [ ] ...

PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
2. Pop
3. Display
4. Exit
Enter your choice: 4
Program ended
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> 
```

Q2. Write a menu driven program with 4 options (Push, Pop, Display, and Exit) to demonstrate the working of stacks using linked-list.

Code:

```
#include <iostream>
using namespace std;
```

```
struct Node
{
    int data;
    Node *next;
};
```

```
Node *top = NULL;
```

```
void push()
{
    int x;

    Node *newNode = new Node;

    cout << "Enter value: ";
    cin >> x;

    newNode->data = x;
    newNode->next = top;
    top = newNode;

    cout << "Value pushed successfully" << endl;
}
```

```
void pop()
```

```

{
    Node *temp;

    if (top == NULL)
    {
        cout << "Stack Underflow" << endl;
    }
    else
    {
        temp = top;

        cout << "Popped value: " << top->data << endl;

        top = top->next;

        delete temp;
    }
}

```

```

void display()
{
    Node *temp;

    if (top == NULL)
    {
        cout << "Stack is empty" << endl;
    }
    else
    {
        temp = top;

        cout << "Stack elements are:" << endl;

        while (temp != NULL)
        {
            cout << temp->data << endl;
            temp = temp->next;
        }
    }
}

```

```

int main()
{
    int choice;

```

```

while (true)
{
    cout << "\n1. Push" << endl;
    cout << "2. Pop" << endl;
    cout << "3. Display" << endl;
    cout << "4. Exit" << endl;

    cout << "Enter your choice: ";
    cin >> choice;

    if (choice == 1)
    {
        push();
    }
    else if (choice == 2)
    {
        pop();
    }
    else if (choice == 3)
    {
        display();
    }
    else if (choice == 4)
    {
        cout << "Program ended" << endl;
        break;
    }
    else
    {
        cout << "Invalid choice" << endl;
    }
}

return 0;
}

```

Output:

```
Terminal
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
```

```
Terminal
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
4. Exit
Enter your choice: 1
Enter value: 7
Value pushed successfully

1. Push
```

```
Terminal
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 2
```

```
Terminal
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
3. Display
4. Exit
Enter your choice: 2
Popped value: 7

1. Push
```

Terminal

Code + - [] [] ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

```
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
```

Terminal

Code + - [] [] ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

```
4. Exit
Enter your choice: 3
Stack is empty
```

```
1. Push
2. Pop
```

Terminal

Code + - [] [] ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

```
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 4
```

Terminal

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

```
2. Pop
3. Display
4. Exit
Enter your choice: 4
Program ended
```

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> [ ]
```

Q3. Write a program to convert infix expression into postfix expression using stack.

Code:

```
#include <iostream>
#include <cctype>
using namespace std;
```

```
char stack[100];
int top = -1;
```

```
void push(char x)
{
    top++;
    stack[top] = x;
}
```

```
char pop()
{
    char x = stack[top];
    top--;
    return x;
}
```

```
}
```

```
int priority(char x)
```

```
{
```

```
    if (x == '^')
```

```
        return 3;
```

```
    if (x == '*' || x == '/')
```

```
        return 2;
```

```
    if (x == '+' || x == '-')
```

```
        return 1;
```

```
    return 0;
```

```
}
```

```
int main()
```

```
{
```

```
    char infix[100];
```

```
    char postfix[100];
```

```
    int i = 0;
```

```
    int j = 0;
```

```
    cout << "Enter infix expression: ";
```

```
    cin >> infix;
```

```
    while (infix[i] != '\0')
```

```
    {
```

```
        if (isdigit(infix[i]))
```

```
        {
```

```
            postfix[j] = infix[i];
```

```
            j++;
```

```
        }
```

```
        else if (infix[i] == '(')
```

```
        {
```

```
            push(infix[i]);
```

```
        }
```

```
        else if (infix[i] == ')')
```

```
        {
```

```
            while (top != -1 && stack[top] != '(')
```

```
            {
```

```
                postfix[j] = pop();
```

```
                j++;
```

```

    }

    pop();
}
else
{
    while (top != -1 && priority(stack[top]) >= priority(infix[i]))
    {
        postfix[j] = pop();
        j++;
    }

    push(infix[i]);
}

i++;
}

while (top != -1)
{
    postfix[j] = pop();
    j++;
}

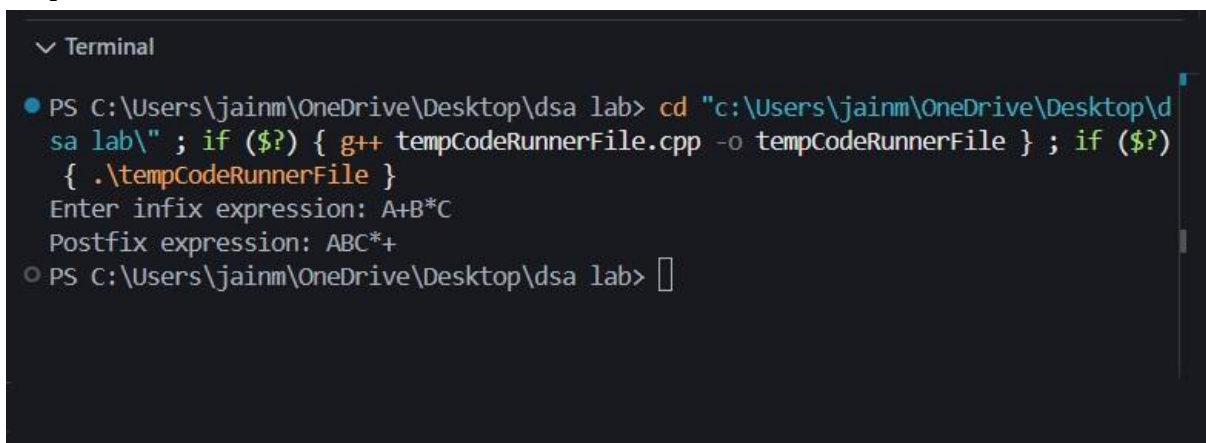
postfix[j] = '\0';

cout << "Postfix expression: " << postfix << endl;

return 0;
}

```

Output:



```

Terminal
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
Enter infix expression: A+B*C
Postfix expression: ABC*+
PS C:\Users\jainm\OneDrive\Desktop\dsa lab>

```

Q4. Write a program to convert infix expression into prefix expression using stack.

Code:

```
#include <iostream>
#include <cstring>
#include <cctype>
using namespace std;

char stack[100];
int top = -1;

void push(char x)
{
    top++;
    stack[top] = x;
}

char pop()
{
    char x = stack[top];
    top--;
    return x;
}

int priority(char x)
{
    if (x == '^')
        return 3;

    if (x == '*' || x == '/')
        return 2;

    if (x == '+' || x == '-')
        return 1;

    return 0;
}

int main()
{
    char infix[100];
    char postfix[100];
    char prefix[100];
```

```

int i, j;
int n;

cout << "Enter infix expression: ";
cin >> infix;

n = strlen(infix);

for (i = 0; i < n / 2; i++)
{
    char temp = infix[i];
    infix[i] = infix[n - i - 1];
    infix[n - i - 1] = temp;
}

for (i = 0; i < n; i++)
{
    if (infix[i] == '(')
        infix[i] = ')';
    else if (infix[i] == ')')
        infix[i] = '(';
}

i = 0;
j = 0;

while (infix[i] != '\0')
{
    if (isalnum(infix[i]))
    {
        postfix[j] = infix[i];
        j++;
    }
    else if (infix[i] == '(')
    {
        push(infix[i]);
    }
    else if (infix[i] == ')')
    {
        while (top != -1 && stack[top] != '(')
        {
            postfix[j] = pop();
            j++;
        }
    }
}

```

```

        pop();
    }
    else
    {
        while (top != -1 && priority(stack[top]) > priority(infix[i]))
        {
            postfix[j] = pop();
            j++;
        }

        push(infix[i]);
    }

    i++;
}

while (top != -1)
{
    postfix[j] = pop();
    j++;
}

postfix[j] = '\0';

n = strlen(postfix);

for (i = 0; i < n; i++)
{
    prefix[i] = postfix[n - i - 1];
}

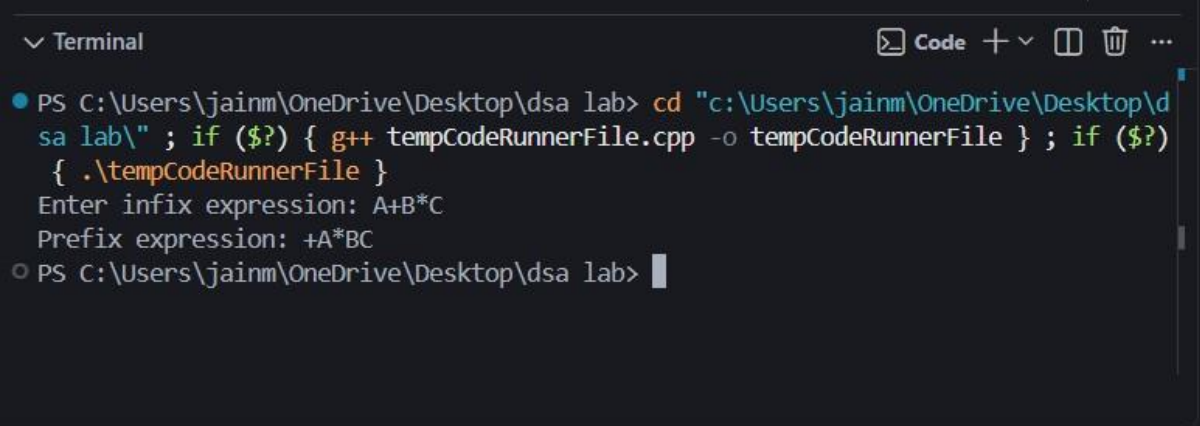
prefix[n] = '\0';

cout << "Prefix expression: " << prefix << endl;

return 0;
}

```

Output:



```
Terminal
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
Enter infix expression: A+B*C
Prefix expression: +A*BC
PS C:\Users\jainm\OneDrive\Desktop\dsa lab>
```

Q5. Write a program to evaluate a postfix expression using stack.

Code:

```
#include <iostream>
#include <cctype>
using namespace std;
```

```
int stack[100];
int top = -1;
```

```
void push(int x)
{
    top++;
    stack[top] = x;
}
```

```
int pop()
{
    int x = stack[top];
    top--;
    return x;
}
```

```
int main()
{
    char postfix[100];

    int i;
    int a, b, result;

    cout << "Enter postfix expression: ";
    cin >> postfix;
```

```

i = 0;

while (postfix[i] != '\0')
{
    if (isdigit(postfix[i]))
    {
        push(postfix[i] - '0');
    }
    else
    {
        b = pop();
        a = pop();

        if (postfix[i] == '+')
            result = a + b;
        else if (postfix[i] == '-')
            result = a - b;
        else if (postfix[i] == '*')
            result = a * b;
        else if (postfix[i] == '/')
            result = a / b;

        push(result);
    }

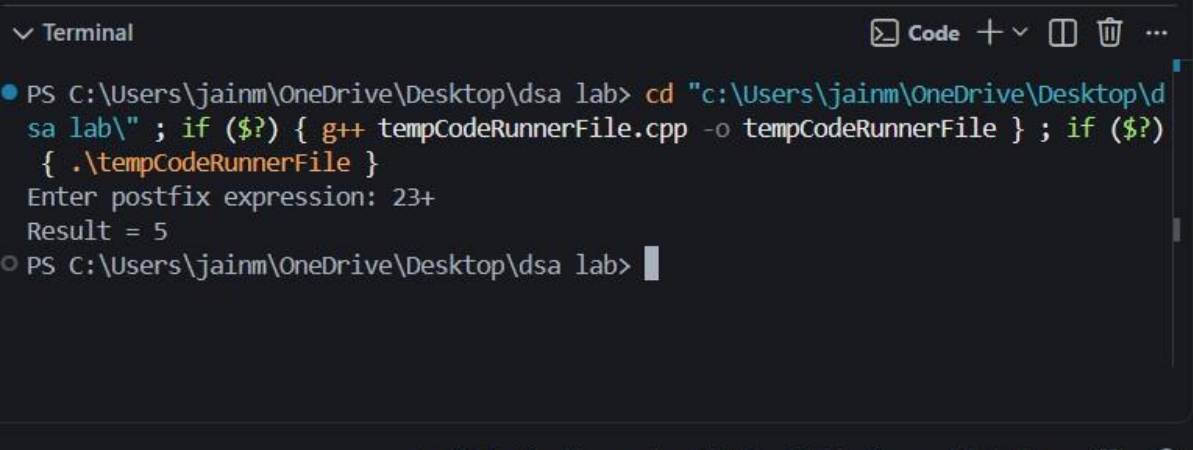
    i++;
}

cout << "Result = " << pop() << endl;

return 0;
}

```

Output:



```
Terminal
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
Enter postfix expression: 23+
Result = 5
PS C:\Users\jainm\OneDrive\Desktop\dsa lab>
```

Ln 62 Col 2 Spaces: 4 UTF-8 CRLF C++ (x) Go Live \$?

Assignment 6

Q1. Write a menu driven program with 4 options (Insert, Delete, Display, and Exit) to demonstrate the working of Queues using arrays.

```
#include <iostream>
using namespace std;

#define SIZE 5

int queue[SIZE];
int front = -1;
int rear = -1;

void insert()
{
    int x;

    if (rear == SIZE - 1)
    {
        cout << "Queue is full\n";
        return;
    }

    cout << "Enter value: ";
    cin >> x;

    if (front == -1)
    {
        front = 0;
    }

    rear++;
    queue[rear] = x;

    cout << "Element inserted\n";
}

void deleteElement()
{
    if (front == -1 || front > rear)
    {
        cout << "Queue is empty\n";
    }
}
```

```

        return;
    }

    cout << "Deleted element: " << queue[front] << endl;

    front++;

    if (front > rear)
    {
        front = -1;
        rear = -1;
    }
}

void display()
{
    if (front == -1)
    {
        cout << "Queue is empty\n";
        return;
    }

    cout << "Queue: ";

    for (int i = front; i <= rear; i++)
    {
        cout << queue[i] << " ";
    }

    cout << endl;
}

int main()
{
    int choice;

    do
    {
        cout << "\n QUEUE MENU \n";
        cout << "1. Insert\n";
        cout << "2. Delete\n";
        cout << "3. Display\n";
        cout << "4. Exit\n";
    }
}

```

```
    cout << "Enter your choice: ";
    cin >> choice;

    switch (choice)
    {
    case 1:
        insert();
        break;

    case 2:
        deleteElement();
        break;

    case 3:
        display();
        break;

    case 4:
        cout << "Program ended\n";
        break;

    default:
        cout << "Wrong choice\n";
    }

} while (choice != 4);

return 0;
}
```

Terminal

Code + ▾

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

QUEUE MENU

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 1

Enter value: 10

Element inserted

QUEUE MENU

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 1

Enter value: 20

Element inserted

QUEUE MENU

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 1

Enter value: 30

Element inserted

Terminal

Code + ▾ □ 🗑️ ...

PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Des

Enter your choice: 1

Enter value: 30

Element inserted

QUEUE MENU

1. Insert

2. Delete

3. Display

4. Exit

Enter your choice: 3

Queue: 10 20 30

QUEUE MENU

1. Insert

2. Delete

3. Display

4. Exit

Enter your choice: 2

Deleted element: 10

QUEUE MENU

1. Insert

2. Delete

3. Display

4. Exit

Enter your choice: 3

Queue: 20 30

QUEUE MENU

1. Insert

2. Delete

3. Display

4. Exit

Enter your choice: 4

Program ended

PS C:\Users\jainm\OneDrive\Desktop\dsa lab>

Q2. Write a menu driven program with 4 options (Insert, Delete, Display, and Exit) to demonstrate the working of Circular Queues (arrays.)

```
#include <iostream>
using namespace std;

#define SIZE 5

int queue[SIZE];
int front = -1;
int rear = -1;

void insert()
{
    int x;

    if ((rear + 1) % SIZE == front)
    {
        cout << "Queue is full\n";
        return;
    }

    cout << "Enter value: ";
    cin >> x;

    if (front == -1)
    {
        front = 0;
        rear = 0;
    }
    else
    {
        rear = (rear + 1) % SIZE;
    }

    queue[rear] = x;

    cout << "Element inserted\n";
}

void deleteElement()
{
    if (front == -1)
    {

```

```

        cout << "Queue is empty\n";
        return;
    }

    cout << "Deleted element: " << queue[front] << endl;

    if (front == rear)
    {
        front = -1;
        rear = -1;
    }
    else
    {
        front = (front + 1) % SIZE;
    }
}

void display()
{
    if (front == -1)
    {
        cout << "Queue is empty\n";
        return;
    }

    cout << "Circular Queue: ";

    int i = front;

    while (true)
    {
        cout << queue[i] << " ";

        if (i == rear)
        {
            break;
        }

        i = (i + 1) % SIZE;
    }

    cout << endl;
}

```

```
int main()
{
    int choice;

    do
    {
        cout << "\nMENU \n";
        cout << "1. Insert\n";
        cout << "2. Delete\n";
        cout << "3. Display\n";
        cout << "4. Exit\n";

        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1:
                insert();
                break;

            case 2:
                deleteElement();
                break;

            case 3:
                display();
                break;

            case 4:
                cout << "Program ended\n";
                break;

            default:
                cout << "Wrong choice\n";
        }

    } while (choice != 4);

    return 0;
}
```

Terminal

Code + - [] [] ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 1

Enter value: 10

Element inserted

MENU

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 1

Enter value: 20

Element inserted

MENU

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 1

Enter value: 30

Element inserted

Terminal

Code + - [] [] ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\d
```

MENU

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 3

Circular Queue: 10 20 30

MENU

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 2

Deleted element: 10

MENU

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 3

Circular Queue: 20 30

MENU

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 1

Enter value: 40

Element inserted

Terminal

Code + - [] [] ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab"
Circular Queue: 20 30
```

MENU

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 1

Enter value: 40

Element inserted

MENU

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 1

Enter value: 50

Element inserted

MENU

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 3

Circular Queue: 20 30 40 50

MENU

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 4

Program ended

PS C:\Users\jainm\OneDrive\Desktop\dsa lab>